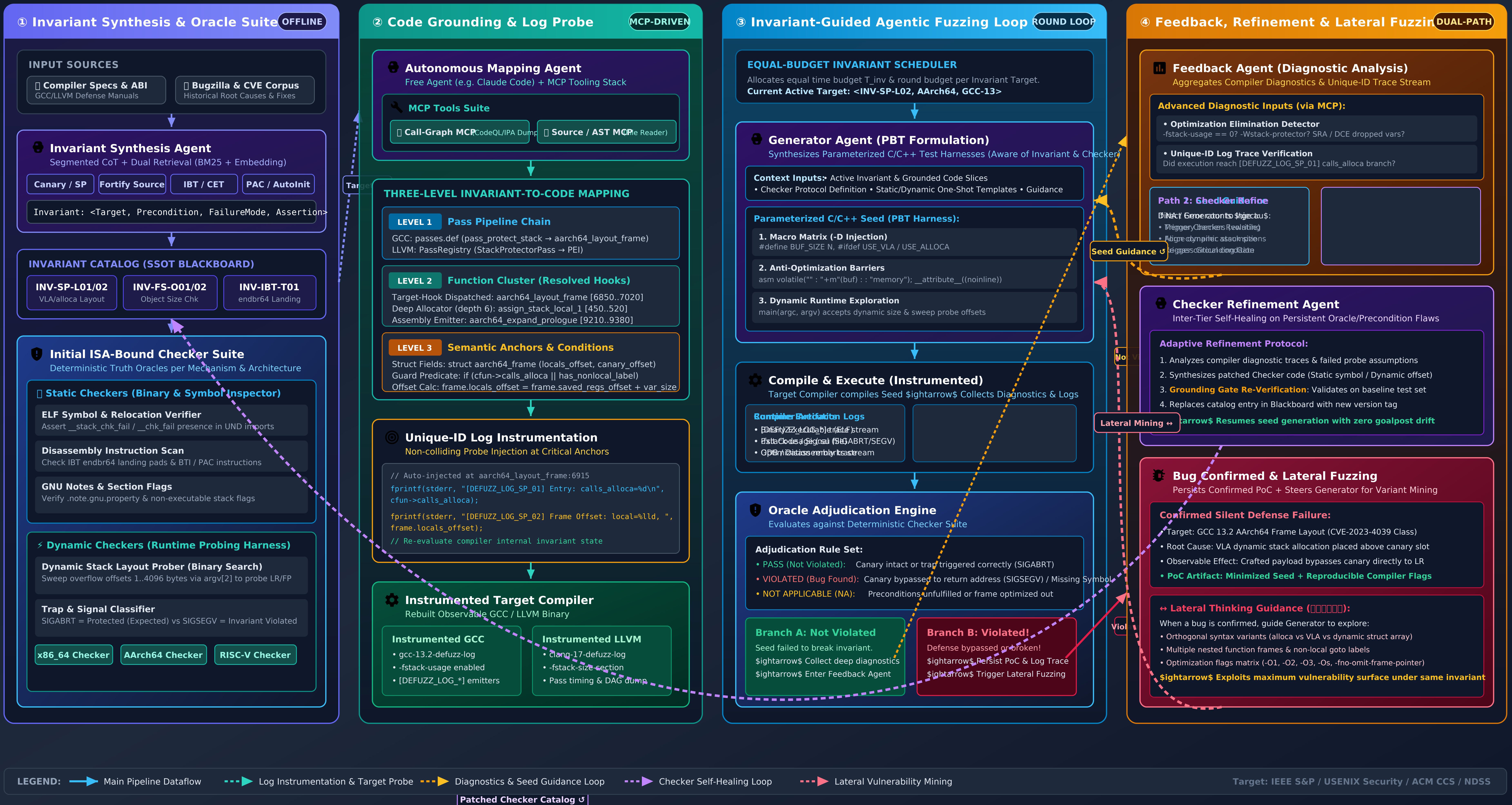
DeFuzz: Invariant-Guided Agentic Testing of Compiler-Instrumented De

PBT Generator Seeds · Unique-ID Log Probes · Compiler Diagnostics · Two-Tier Refinement Loop · Lateral Bug Fuzzing

Autonomous Invariant Distillation · Code Grounding & Log Probe · PBT Parameterized Fuzzing · Compiler Diagnostics & Dual-Path Adaptation



① Invariant Synthesis & Oracle Suite — OFFLINE

INPUT SOURCES

- **Compiler Specs & ABI** — GCC/LLVM Defense Manuals
- **Bugzilla & CVE Corpus** — Historical Root Causes & Fixes

Invariant Synthesis Agent

Segmented CoT + Dual Retrieval (BM25 + Embedding)

Canary / SP · Fortify Source · IBT / CET · PAC / AutoInit

`Invariant: <Target, Precondition, FailureMode, Assertion>`

INVARIANT CATALOG (SSOT BLACKBOARD)

- **INV-SP-L01/02** — VLA/alloca Layout
- **INV-FS-O01/02** — Object Size Chk
- **INV-IBT-T01** — endbr64 Landing

Initial ISA-Bound Checker Suite

Deterministic Truth Oracles per Mechanism & Architecture

Static Checkers (Binary & Symbol Inspector)

- **ELF Symbol & Relocation Verifier** — Assert __stack_chk_fail / __chk_fail presence in UND imports
- **Disassembly Instruction Scan** — Check IBT endbr64 landing pads & BTI / PAC instructions
- **GNU Notes & Section Flags** — Verify .note.gnu.property & non-executable stack flags

⚡ Dynamic Checkers (Runtime Probing Harness)

- **Dynamic Stack Layout Prober (Binary Search)** — Sweep overflow offsets 1..4096 bytes via argv[2] to probe LR/FP
- **Trap & Signal Classifier** — SIGABRT = Protected (Expected) vs SIGSEGV = Invariant Violated

x86_64 Checker · AArch64 Checker · RISC-V Checker

② Code Grounding & Log Probe — MCP-DRIVEN

Autonomous Mapping Agent

Free Agent (e.g. Claude Code) + MCP Tooling Stack

MCP Tools Suite

- **Call-Graph MCP** (CodeQL/IPA Dump
- **Source / AST MCP** (File Reader)

THREE-LEVEL INVARIANT-TO-CODE MAPPING

LEVEL 1 — Pass Pipeline Chain

GCC: passes.def (pass_protect_stack → aarch64_layout_frame)
LLVM: PassRegistry (StackProtectorPass → PEI)

LEVEL 2 — Function Cluster (Resolved Hooks)

Target-Hook Dispatched: aarch64_layout_frame [6850..7020]
Deep Allocator (depth 6): assign_stack_local_1 [450..520]
Assembly Emitter: aarch64_expand_prologue [9210..9380]

LEVEL 3 — Semantic Anchors & Conditions

Struct Fields: struct aarch64_frame (locals_offset, canary_offset)
Guard Predicate: if (cfun->calls_alloca || has_nonlocal_label)
Offset Calc: frame.locals_offset = frame.saved_regs_offset + var_size

Unique-ID Log Instrumentation

Non-colliding Probe Injection at Critical Anchors

```
// Auto-injected at aarch64_layout_frame:6915
fprintf(stderr, "[DEFUZZ_LOG_SP_01] Entry: calls_alloca=%d\n",
cfun->calls_alloca);

fprintf(stderr, "[DEFUZZ_LOG_SP_02] Frame Offset: local=%lld, ",
frame.locals_offset);
// Re-evaluate compiler internal invariant state
```

Instrumented Target Compiler

Rebuilt Observable GCC / LLVM Binary

Instrumented GCC

- gcc-13.2-defuzz-log
- -fstack-usage enabled
- [DEFUZZ_LOG_*] emitters

Instrumented LLVM

- clang-17-defuzz-log
- -fstack-size-section
- Pass timing & DAG dump

③ Invariant-Guided Agentic Fuzzing Loop — ROUND LOOP

EQUAL-BUDGET INVARIANT SCHEDULER

Allocates equal time budget T_inv & round budget per Invariant Target.
Current Active Target: <INV-SP-L02, AArch64, GCC-13>

Generator Agent (PBT Formulation)

Synthesizes Parameterized C/C++ Test Harnesses (Aware of Invariant & Checker)

Context Inputs: • Active Invariant & Grounded Code Slices
• Checker Protocol Definition • Static/Dynamic One-Shot Templates • Guidance

Parameterized C/C++ Seed (PBT Harness):

1. **Macro Matrix (-D Injection)** — #define BUF_SIZE N, #ifdef USE_VLA / USE_ALLOCA
2. **Anti-Optimization Barriers** — asm volatile("" : "+m"(buf) : : "memory"); __attribute__((noinline))
3. **Dynamic Runtime Exploration** — main(argc, argv) accepts dynamic size & sweep probe offsets

Compile & Execute (Instrumented)

Target Compiler compiles Seed $ightarrow$ Collects Diagnostics & Logs

Compiler Artifacts / Runtime Execution Logs

- Binary Executable (ELF) / [DEFUZZ_LOG_*] trace stream
- -fstack-usage (.su file) / Exit Code / Signal (SIGABRT/SEGV)
- Optimization remarks / GDB / Disassembly trace

Oracle Adjudication Engine

Evaluates against Deterministic Checker Suite

Adjudication Rule Set:

- PASS (Not Violated): Canary intact or trap triggered correctly (SIGABRT)
- VIOLATED (Bug Found): Canary bypassed to return address (SIGSEGV) / Missing Symbol
- NOT APPLICABLE (NA): Preconditions unfulfilled or frame optimized out

Branch A: Not Violated

Seed failed to break invariant.
$ightarrow$ Collect deep diagnostics
$ightarrow$ Enter Feedback Agent

Branch B: Violated!

Defense bypassed or broken!
$ightarrow$ Persist PoC & Log Trace
$ightarrow$ Trigger Lateral Fuzzing

④ Feedback, Refinement & Lateral Fuzzin — DUAL-PATH

Feedback Agent (Diagnostic Analysis)

Aggregates Compiler Diagnostics & Unique-ID Trace Stream

Advanced Diagnostic Inputs (via MCP):

- **Optimization Elimination Detector** — -fstack-usage == 0? -Wstack-protector? SRA / DCE dropped vars?
- **Unique-ID Log Trace Verification** — Did execution reach [DEFUZZ_LOG_SP_01] calls_alloca branch?

Path 1: Seed Guidance / Path 2: Checker Refine

If NA / Direct Generator to $ightarrow$:

- Trigger Checker Rewriting / Memory barriers (volatile)
- Force compiler assumptions / Align dynamic stack sizes
- Trigger critical conditions / Re-pass Grounding Gate

Checker Refinement Agent

Inter-Tier Self-Healing on Persistent Oracle/Precondition Flaws

Adaptive Refinement Protocol:

1. Analyzes compiler diagnostic traces & failed probe assumptions
2. Synthesizes patched Checker code (Static symbol / Dynamic offset)
3. **Grounding Gate Re-Verification**: Validates on baseline test set
4. Replaces catalog entry in Blackboard with new version tag

arrow$ Resumes seed generation with zero goalpost drift

Bug Confirmed & Lateral Fuzzing

Persists Confirmed PoC + Steers Generator for Variant Mining

Confirmed Silent Defense Failure:

- Target: GCC 13.2 AArch64 Frame Layout (CVE-2023-4039 Class)
- Root Cause: VLA dynamic stack allocation placed above canary slot
- Observable Effect: Crafted payload bypasses canary directly to LR
- **PoC Artifact: Minimized Seed + Reproducible Compiler Flags**

↔ Lateral Thinking Guidance (□□□□□□):

When a bug is confirmed, guide Generator to explore:

- Orthogonal syntax variants (alloca vs VLA vs dynamic struct array)
- Multiple nested function frames & non-local goto labels
- Optimization flags matrix (-O1, -O2, -O3, -Os, -fno-omit-frame-pointer)

$ightarrow$ Exploits maximum vulnerability surface under same invariant

LEGEND: Main Pipeline Dataflow · Log Instrumentation & Target Probe · Diagnostics & Seed Guidance Loop · Checker Self-Healing Loop · Lateral Vulnerability Mining

Target: IEEE S&P / USENIX Security / ACM CCS / NDSS